

- Introduction to HTML
- HTML Document Structure
- HTML Elements & Tags
- Headings
- Paragraphs
- Font & Formatting Tags
- Attributes
- Lists
- Tables
- Links (Anchor Tag)
- Images
- Forms
- Semantic Elements
- Multimedia (Audio/Video)
- HTML Entities
- Iframes
- Comments and Favicon
- HTML File Path
- Responsive Design



1. Introduction to HTML

- **HTML** stands for **HyperText Markup Language**.
- It is the standard markup language used to create and design web pages.
- HTML is not a programming language; it is a **markup language** that describes the structure and layout of a web document.
- HTML files are saved with a `.html` or `.htm` extension and can be viewed in any web browser.
- HTML works together with **CSS** (for styling) and **JavaScript** (for interactivity) to build complete web applications.

History of HTML

- HTML was first developed by Tim Berners-Lee in 1991

- HTML was first developed by Tim Berners-Lee in 1991.

Timeline of HTML Versions

Version	Year Released	Key Features / Notes
HTML 1.0	1993	First official release; basic tags for text, links
HTML 2.0	1995	Standardized existing features; forms introduced
HTML 3.2	1997	Tables, applets, scripting (JavaScript) support
HTML 4.0	1997	CSS support, richer forms, deprecated older tags
HTML 4.01	1999	Minor bug fixes, strict/loose/frameset DOCTYPEs
XHTML 1.0	2000	XML-based HTML, stricter syntax
HTML5	2014	Semantic elements, audio/video, APIs, mobile support

Tip: HTML5 is the current standard, but browsers support older versions for compatibility.

How to Execute or View an HTML File

1. **Create an HTML file:**
   Save your HTML code in a file with a `.html` extension, for example, `index.html`.
2. **Open in a Web Browser:**
   - Double-click the HTML file, or
   - Right-click and choose "Open with" and select your preferred browser (Chrome, Firefox, Edge, etc.).
3. **Using a Code Editor:**
   Most code editors (like VS Code, Sublime Text) allow you to right-click the file and select "Open with Live Server" or similar options for live preview.

Note:
HTML files are executed (rendered) by the browser, not "run" like a program. The browser interprets the HTML and displays the web page accordingly.

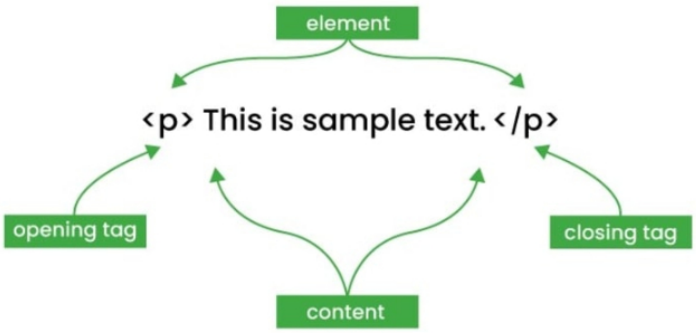
2. HTML Document Structure

```html
<!DOCTYPE html>
<html>
    <head>
        <title>Page Title</title>
    </head>
    <body>
        <!-- Content goes here -->
    </body>
</html>
```

3. HTML Elements and Tags

What are HTML Elements and Tags?

HTML documents are made up of **elements**, which define the structure and content of a web page. An **element** typically consists of a **start tag**, **content**, and an **end tag**.



Basic Syntax

```
<tagname>Content goes here</tagname>
```

- **Start tag:** `<tagname>`
- **Content:** The information or other elements inside the tags
- **End tag:** `</tagname>`

Example

```
<p>This is a paragraph.</p>
```

Types of Elements

1. `Self-Closing` (Void) Elements

Some elements do not have any content or closing tag. These are called **self-closing** or **void elements**.

Examples:

```
<img src="image.jpg" alt="Description">
<br>
<hr>
<input type="text">
```

2. `Nesting` Elements

Elements can be **nested** inside other elements:

```
<div>
    <h1>Title</h1>
    <p>This is a paragraph inside a div.</p>
</div>
```

3. `Block-level` vs `Inline` Elements

- **Block-level elements**: Start on a new line and take up the full width available. Examples: `<div>`, `<p>`, `<h1>`, `<ul>`.
- **Inline elements**: Do not start on a new line and only take up as much width as necessary. Examples: `<span>`, `<a>`, `<img>`.

Case Sensitivity

HTML tags are **not case-sensitive**: `<h1>` is the same as `<H1>`, but lowercase is recommended.

Some examples of HTML tags:

Tag Example	Description
`<h1>Title</h1>`	Heading
`<p>Text</p>`	Paragraph
`<a href="#">Link</a>`	Hyperlink
`<img src="..." />`	Image (self-closing)

Tip: Always properly close your tags and nest them correctly for valid HTML.

4. Headings

HTML provides six levels of headings, from `<h1>` (the most important) to `<h6>` (the least important):

- `<h1>`: Main heading (used for the title or most important heading)
- `<h2>`: Subheading (used for major sections)
- `<h3>`: Sub-subheading (used for subsections)
- `<h4>`, `<h5>`, `<h6>`: Used for further subdivisions

Syntax

```
<h1>This is a Heading 1</h1>
<h2>This is a Heading 2</h2>
<h3>This is a Heading 3</h3>
<h4>This is a Heading 4</h4>
<h5>This is a Heading 5</h5>
<h6>This is a Heading 6</h6>
```

Key Points

- Headings help structure your content and improve accessibility and SEO.
- Only one `<h1>` should be used per page, typically for the main title.
- Headings are block-level elements and automatically add space above and below.
- Browsers display headings in bold and with decreasing font size from `<h1>` to `<h6>`.

Tip: Use headings in a logical order to create a clear document outline.

5. Paragraphs

- Use `<p>` tag for paragraphs.
- Paragraphs are block-level elements
- They contain inline elements

Syntax:

Syntax:

```
<p>This is a paragraph.</p>
```

Common Attributes

Attribute	Description	Example Usage
`align`	Text alignment [`left|center|right|justify`] (deprecated, use CSS)	`<p align="center">Text</p>`

6. Font & Formating tags

Font Tag (`<font>`) — Deprecated

The `<font>` tag was used in older versions of HTML to specify the font face, size, and color of text. However, it is **deprecated in HTML5** and should not be used in modern web development. Instead, use CSS for all font styling.

Syntax (Deprecated)

```
<font face="Arial" size="4" color="red">This is some text</font>
```

- `face` : Sets the font family (e.g., Arial, Verdana).
- `size` : Sets the font size (1–7, not recommended).
- `color` : Sets the text color.

Modern Alternative (CSS)

Use CSS for font styling:

```
<span style="font-family: Arial; font-size: 20px; color: red;">This is some text</span>
```

Formatting Tags

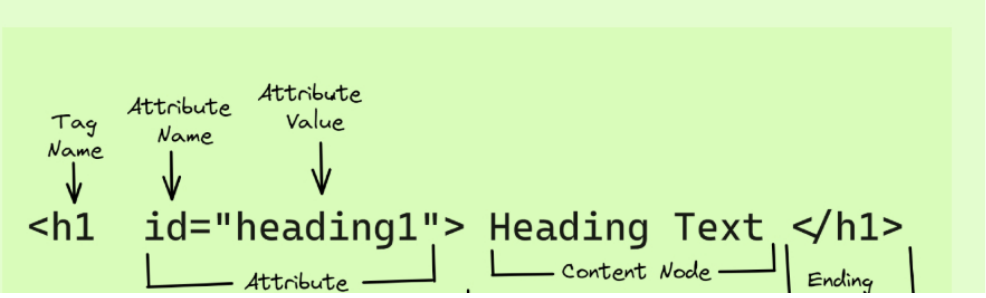
- **Bold:** `<strong>Text</strong>` or `<b>Text</b>`
- **Italic:** `<em>Text</em>` or `<i>Text</i>`
- **Underline:** `<u>Text</u>`
- **Strikethrough:** `<s>Text</s>` or `<del>Text</del>`
- **Inserted Text:** `<ins>Text</ins>`
- **Subscript:** `<sub>Text</sub>`
- **Superscript:** `<sup>Text</sup>`
- **Marked/Highlighted:** `<mark>Text</mark>`
- **Small Text:** `<small>Text</small>`
- **Code:** `<code>Text</code>`
- **Keyboard Input:** `<kbd>Text</kbd>`
- **Sample Output:** `<samp>Text</samp>`
- **Quotation:** `<q>Text</q>` (inline), `<blockquote>Text</blockquote>` (block)
- **Abbreviation:** `<abbr title="Full Text">Abbr</abbr>`
- **Citation:** `<cite>Text</cite>`
- **Definition:** `<dfn>Text</dfn>`
- **Bidirectional Override:** `<bdo dir="rtl">Text</bdo>`
- **Details and Summary:** `<details><summary>Title</summary>Content</details>`

7. Attributes

HTML **attributes** provide additional information about HTML elements. They are always included in the opening tag and usually come in name/value pairs like `name="value"` .

Key Points about Attributes

- Attributes modify the default functionality or appearance of an element.
- Most attributes have a value, enclosed in double or single quotes.
- Some attributes are **global** (can be used on any element), while others are specific to certain elements.





Basic Syntax

```
<tagname attribute="value">Content</tagname>
```

Example

```
<a href="https://www.example.com" target="_blank">Visit Example</a>
```

- `href` is an attribute of the `<a>` tag, specifying the link's destination.
- `target="_blank"` is another attribute, specifying that the link opens in a new tab.

Global Attributes in HTML

Global attributes can be used on **any** HTML element (unless otherwise noted).

Common HTML Attributes

Attribute	Description	Example Usage
`id`	Unique identifier for the element	`<div id="main"></div>`
`class`	Assigns one or more class names	`<p class="intro"></p>`
`style`	Inline CSS styling	`<h1 style="color:blue;"></h1>`
`src`	Image or media source	`<img src="logo.png">`
`alt`	Alternative text for images	`<img alt="Description">`
`href`	Link destination (for `<a>`)	`<a href="url"></a>`
`type`	Input or button type	`<input type="text">`
`value`	Specifies value of input, button, etc.	`<input value="Submit">`
`disabled`	Disables an element	`<button disabled>Click</button>`
`checked`	Pre-selects checkbox/radio	`<input type="checkbox" checked>`
`title`	Tooltip text shown on hover	`<img title="Logo">`
`hidden`	Hides the element from view	`<div hidden>Secret</div>`
`tabindex`	Sets tab order for keyboard navigation	`<button tabindex="1">Click</button>`
`draggable`	Specifies if element is draggable (`true` / `false`)	`<img draggable="true">`
`contenteditable`	Makes element content editable by user	`<div contenteditable="true">Edit me</div>`
`lang`	Language of element's content	`<p lang="fr">Bonjour</p>`
`dir`	Text direction (`ltr`, `rtl`, `auto`)	`<p dir="rtl">مرحبا</p>`
`data-*`	Custom data attributes for storing extra info	`<div data-user-id="42"></div>`
`accesskey`	Keyboard shortcut to focus/activate element	`<button accesskey="s">Save</button>`
`spellcheck`	Enables/disables spell checking (`true` / `false`)	`<textarea spellcheck="false"></textarea>`
`aria-*`	Accessibility attributes (ARIA roles, labels, etc.)	`<button aria-label="Close"></button>`
`autofocus`	Automatically focuses the element on page load	`<input autofocus>`
`bgcolor`	Background color (deprecated, use CSS)	`<body bgcolor="lightblue">`
`charset`	Character encoding for `<meta>` tag	`<meta charset="UTF-8">`
`background`	Background image (deprecated, use CSS)	`<body background="bg.jpg">`
`leftmargin`	Left margin space (deprecated, use CSS)	`<body leftmargin="20">`

Attribute Rules

- Attribute names are **not case-sensitive** (`HREF` is the same as `href`).
- Always use quotes around attribute values (recommended).
- Some attributes (like `disabled`, `checked`, `readonly`) are **boolean** and don't require a value.

Example: Anchor Tag with Attributes

```
<a href="https://www.example.com" target="_blank" title="Go to Example">Visit</a>
```

- `href` : URL to visit
- `target="_blank"` : Opens link in new tab
- `title` : Tooltip shown on hover

Tip: Use meaningful attribute values to improve accessibility and SEO.

Example with multiple global attributes:

```
<input
    id="email"
    class="input-field"
    type="email"
    placeholder="Enter your email"
    tabindex="2"
    aria-label="Email Address"
    required
>
```

Tip: Use global attributes to improve accessibility, interactivity, and maintainability of your HTML.

8. Lists

HTML supports three main types of lists:

1. Ordered List (`<ol>`)

- Displays items in a numbered sequence.

```
<ol>
    <li>First item</li>
    <li>Second item</li>
    <li>Third item</li>
</ol>
```

2. Unordered List (`<ul>`)

- Displays items with bullets.

```
<ul>
    <li>Item one</li>
    <li>Item two</li>
    <li>Item three</li>
</ul>
```

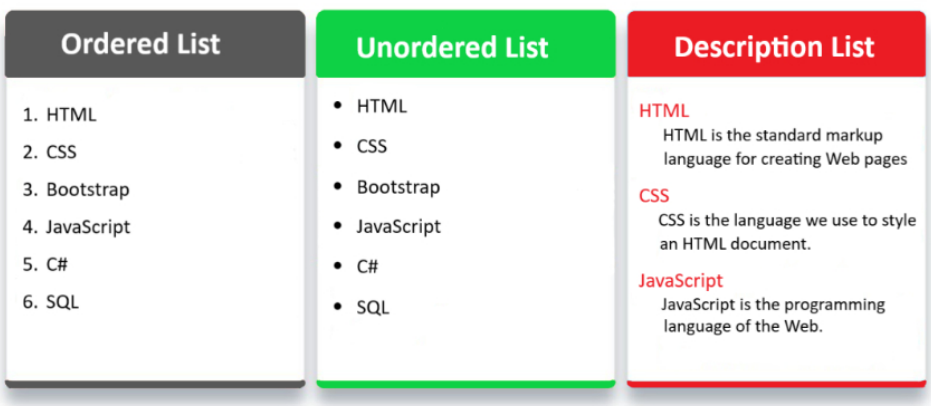
3. Description List (`<dl>`)

- Used for terms and their descriptions.

```
<dl>
    <dt>HTML</dt>
    <dd>HyperText Markup Language</dd>
    <dt>CSS</dt>
    <dd>Cascading Style Sheets</dd>
</dl>
```

Key Points:

- Use `<li>` for each item in ordered and unordered lists.
- Lists can be nested inside each other.
- Description lists use `<dt>` (term) and `<dd>` (description).



Below are common HTML attributes for lists, organized in a table:

Ordered List (`<ol>`) Attributes

Attribute	Use/Element	Description
`type`	`<ol>`	Sets marker type: `1` (default), `A`, `a`, `I`, `i`
`start`	`<ol>`	Sets starting value for first item (e.g., `start="5"`)
`reversed`	`<ol>`	Reverses the numbering order

Unordered List (`<ul>`) Attributes

Attribute	Use/Element	Description
`type`	`<ul>` (deprecated)	Sets bullet style: `disc`, `circle`, `square` (not recommended)

List Item (`<li>`) Attributes

Attribute	Use/Element	Description
`value`	`<li>` (in `<ol>`)	Sets specific value for a list item (overrides automatic number)

Tip: For advanced list styling, use CSS with the `list-style-type` property.

Nested Lists

- Lists can be nested within each other to create sub-items.

```
<ol>
    <li>Item One
        <ul>
            <li>Sub-item One</li>
            <li>Sub-item Two</li>
        </ul>
    </li>
    <li>Item Two</li>
</ol>
```

9. Tables

HTML tables are used to display tabular data in rows and columns.

Basic Table Structure

```
<table>
    <tr>
        <th>Header 1</th>
        <th>Header 2</th>
    </tr>
    <tr>
        <td>Data 1</td>
        <td>Data 2</td>
    </tr>
</table>
```

- `<table>`: Defines the table.
- `<tr>`: Table row.
- `<th>`: Table header cell (bold and centered by default).
- `<td>`: Table data cell.

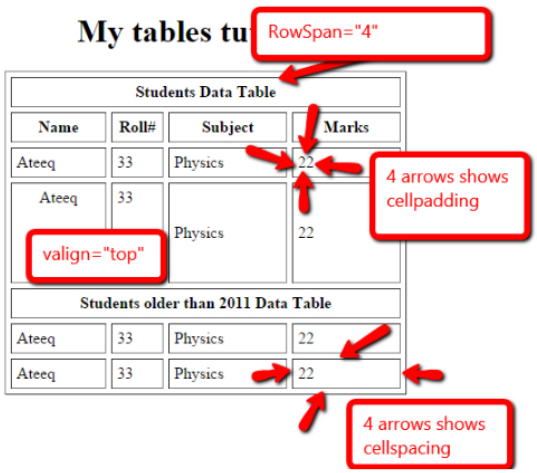


Table Elements

Element	Description
`<table>`	Container for all table content
`<tr>`	Table row
`<th>`	Table header cell
`<td>`	Table data cell
`<thead>`	Groups header content
`<tbody>`	Groups body content
`<tfoot>`	Groups footer content
`<caption>`	Table title/caption
`<colgroup>`	Groups columns for styling
`<col>`	Specifies column properties within `<colgroup>`

Example with Table Sections

```
<table>
    <caption>Student Grades</caption>
    <colgroup>
        <col span="1" style="background-color: #f2f2f2;">
        <col span="1" style="background-color: #e6f7ff;">
    </colgroup>
    <thead>
        <tr>
            <th>Name</th>
            <th>Grade</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Alice</td>
            <td>A</td>
        </tr>
        <tr>
            <td>Bob</td>
            <td>B</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <td colspan="2">End of List</td>
        </tr>
    </tfoot>
</table>
```

- `<colgroup>` groups columns for styling.
- `<col>` applies styles to individual columns (here, background colors).

Table Attributes

Attribute	Element(s)	Description
`border`	`<table>`	Sets border width (use CSS instead)
`colspan`	`<td>`, `<th>`	Merges cells horizontally
`rowspan`	`<td>`, `<th>`	Merges cells vertically
`scope`	`<th>`	Defines header scope (`row`, `col`, etc.)
`align`	`<table>`, `<tr>`, `<td>`, `<th>`	Aligns content (deprecated, use CSS)
`width`	`<table>`, `<td>`, `<th>`	Sets width (use CSS instead)
`height`	`<table>`, `<td>`, `<th>`	Sets height (use CSS instead)
`cellpadding`	`<table>`	Sets space between cell content and border (use CSS)
`cellspacing`	`<table>`	Sets space between cells (use CSS)
`bgcolor`	`<table>`, `<tr>`, `<td>`, `<th>`	Sets background color (use CSS instead)
`valign`	`<td>`, `<th>`	Vertical alignment of content (deprecated, use CSS)

Example with `colspan` and `rowspan`:

```
<table>
    <tr>
        <th rowspan="2">Name</th>
        <th colspan="2">Scores</th>
    </tr>
    <tr>
        <td>Math</td>
        <td>Science</td>
    </tr>
    <tr>
        <td>John</td>
        <td>90</td>
```

```
        <td>85</td>
    </tr>
</table>
```

Tips:

- Use `<th>` for headers to improve accessibility.
- Use `<caption>` for table titles.
- Use `<thead>`, `<tbody>`, and `<tfoot>` for better structure and styling.

10. Links (Anchor Tag)

The **anchor tag** (`<a>`) is used to create hyperlinks, allowing users to navigate to other web pages, files, locations within the same page, email addresses, or any URL.

- `href` : Specifies the destination URL or resource.

Examples

1. Link to Another Website

```
<a href="https://www.google.com">Visit Google</a>
```

2. Open Link in a New Tab

```
<a href="https://www.example.com" target="_blank">Open in New Tab</a>
```

- `target="_blank"` : Opens the link in a new browser tab or window.

3. Link to an Email Address

```
<a href="mailto:someone@example.com">Send Email</a>
```

4. Link to a Phone Number

```
<a href="tel:+1234567890">Call Us</a>
```

5. Link to a File (Download)

```
<a href="files/sample.pdf" download>Download PDF</a>
```

- `download` : Suggests the browser download the file instead of navigating to it.

6. Link to a Section on the Same Page (Anchor Link)

```
<a href="#section2">Go to Section 2</a>

<!-- Later in the page -->
<h2 id="section2">Section 2</h2>
```

- `href="#id"` : Jumps to the element with the matching `id` .

Common Attributes

Attribute	Description
`href`	Destination URL or resource
`target`	Where to open the link (`_self` , `_blank` , `_parent` , `_top`)
`title`	Tooltip text shown on hover
`download`	Prompts file download
`rel`	Specifies relationship (e.g., `noopener` , `noreferrer` , `nofollow`)
`id`	Unique identifier (for anchor links)

Example with Multiple Attributes

```
<a href="https://github.com" target="_blank" rel="noopener noreferrer" title="GitHub Homepage">GitHub</a>
```

Key Points

- Always use meaningful link text for accessibility.
- Use `rel="noopener noreferrer"` with `target="_blank"` for security.
- The anchor tag can wrap images or other elements to make them clickable.

Tip: Use anchor tags for navigation, downloads, email, phone links, and in-page navigation.

11. Images & Marquee

Images are added to HTML documents using the `<img>` tag. This tag is self-closing and does not require a closing tag.

Basic Syntax

```
<img src="image-url" alt="Description">
```

- `src` : Specifies the path or URL to the image file.
- `alt` : Provides alternative text if the image cannot be displayed (important for accessibility and SEO).

Example

```
<img src="logo.png" alt="Company Logo" width="200" height="100">
```

- `width` and `height` (optional): Set the display size of the image in pixels.

Common Attributes

Attribute	Description	Example Usage
`src`	Path or URL to the image	`<img src="cat.jpg">`
`alt`	Alternative text for screen readers or if image fails to load	`<img alt="A cute cat">`
`width`	Width of the image (in pixels or %)	`<img width="300">`
`height`	Height of the image (in pixels or %)	`<img height="200">`
`title`	Tooltip text shown on hover	`<img title="Cat Photo">`
`loading`	Lazy loading (`lazy` , `eager` , `auto`)	`<img loading="lazy">`
`srcset`	Responsive images (different images for different screen sizes)	`<img srcset="small.jpg 500w, large.jpg 1000w">`
`sizes`	Specifies sizes for responsive images	`<img sizes="(max-width: 600px) 100vw, 50vw">`
`hspace`	Horizontal space around the image (deprecated)	`<img hspace="10">`
`vspace`	Vertical space around the image (deprecated)	`<img vspace="10">`

Responsive Images

Use `srcset` and `sizes` for responsive images:

```
<img
    src="small.jpg"
    srcset="small.jpg 500w, medium.jpg 1000w, large.jpg 1500w"
    sizes="(max-width: 600px) 500px, (max-width: 1200px) 1000px, 1500px"
    alt="Responsive Example">
```

Image as a Link

Wrap the `<img>` tag inside an `<a>` tag to make the image clickable:

```
<a href="https://www.example.com">
    <img src="logo.png" alt="Visit Example">
</a>
```

Best Practices

- Always include the `alt` attribute for accessibility.
- Use descriptive `alt` text.
- Optimize image size for faster loading.
- Use modern formats like WebP for better performance when possible.
- Use `loading="lazy"` for images below the fold to improve page speed.

Tip: Use CSS for advanced image styling (borders, shadows, rounded corners, etc.).

Marquee Tag (`<marquee>`) — Deprecated The `<marquee>` tag was used to create scrolling text or images across the screen.

Basic Syntax

```
<marquee>Scrolling text goes here</marquee>
```

Common Attributes:

Attribute	Description	Example Value
`behavior`	Scrolling type: `scroll` , `slide` , `alternate`	`behavior="scroll"`
`direction`	Scroll direction: `left` , `right` , `up` , `down`	`direction="left"`
`scrollamount`	Speed of scrolling (higher = faster)	`scrollamount="6"`
`scrolldelay`	Delay between each scroll movement (in ms)	`scrolldelay="85"`

`scrolldelay`	Delay between each scroll movement (in ms)	`scrolldelay="85"`
`loop`	Number of times to loop (`-1` or `infinite` for endless)	`loop="3"`

Note: The `<marquee>` tag is obsolete. Use CSS for scrolling effects in modern projects.

12. Forms

HTML **forms** are used to collect user input and send it to a server for processing. Forms can include text fields, checkboxes, radio buttons, dropdowns, buttons, and more.

Basic Structure

```
<form action="submit-url" method="post">
    <!-- Form elements go here -->
</form>
```

- `action` : URL where form data is sent.
- `method` : HTTP method (`get` or `post`).

Common Form Elements

Element	Description	Example
`<input>`	Single-line input (text, password, etc.)	`<input type="text" name="username">`
`<textarea>`	Multi-line text input	`<textarea name="message"></textarea>`
`<select>`	Dropdown list	`<select name="country">...</select>`
`<option>`	Option in dropdown	`<option value="IN">India</option>`
`<button>`	Clickable button	`<button type="submit">Submit</button>`
`<label>`	Label for input	`<label for="email">Email:</label>`
`<fieldset>`	Groups related elements	`<fieldset>...</fieldset>`
`<legend>`	Caption for `<fieldset>`	`<legend>Personal Info</legend>`

Input Types

- `text` , `password` , `email` , `number` , `checkbox` , `radio` , `date` , `file` , `submit` , `reset` , etc.

Example Inputs

```
<form action="/submit" method="post">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email">

    <label for="age">Age:</label>
    <input type="number" id="age" name="age" min="1" max="120">

    <label>
        <input type="checkbox" name="subscribe" checked>
        Subscribe to newsletter
    </label>

    <label>Gender:</label>
    <input type="radio" id="male" name="gender" value="male">
    <label for="male">Male</label>
    <input type="radio" id="female" name="gender" value="female">
    <label for="female">Female</label>

    <label for="country">Country:</label>
    <select id="country" name="country">
        <option value="IN">India</option>
        <option value="US">USA</option>
    </select>

    <label for="message">Message:</label>
    <textarea id="message" name="message"></textarea>

    <button type="submit">Submit</button>
    <button type="reset">Reset</button>
</form>
```

Attributes

Attribute	Description
`name`	Name of the input (used as key in form data)
`value`	Default value
`placeholder`	Hint text inside input
`required`	Makes field mandatory

`readonly`	Field cannot be edited
`disabled`	Field is disabled
`checked`	Checkbox/radio is pre-selected
`min`, `max`	Minimum/maximum value (number/date)
`maxlength`	Maximum number of characters
`pattern`	Regex pattern for validation

Form Submission

- **GET**: Appends form data to URL (visible, for search/filter forms).
- **POST**: Sends data in HTTP body (secure, for sensitive data).

Example: Simple Login Form

```
<form action="/login" method="post">
    <label for="user">Username:</label>
    <input type="text" id="user" name="user" required>

    <label for="pass">Password:</label>
    <input type="password" id="pass" name="pass" required>

    <button type="submit">Login</button>
</form>
```

Best Practices

- Always use `<label>` for accessibility.
- Use `required`, `type`, and `pattern` for client-side validation.
- Group related fields with `<fieldset>` and `<legend>`.
- Use semantic input types (`email`, `number`, etc.) for better UX.

Tip: Use CSS to style forms for better appearance and usability.

13. Semantic Elements

Semantic HTML refers to the use of HTML tags that convey the meaning and structure of content, rather than just its appearance. Semantic elements clearly describe their purpose both to the browser and to developers, improving accessibility, SEO, and code readability.

Why Use Semantic HTML?

- **Accessibility:** Screen readers and assistive technologies can better interpret the content.
- **SEO:** Search engines understand the structure and importance of content, improving rankings.
- **Maintainability:** Code is easier to read, understand, and maintain.
- **Standardization:** Follows best practices and web standards.

Common Semantic Elements

Element	Description
`<header>`	Introductory content or navigation links
`<nav>`	Navigation links/menus
`<main>`	Main content unique to the page
`<section>`	Thematic grouping of content
`<article>`	Independent, self-contained content
`<aside>`	Content tangentially related (sidebars, etc.)
`<footer>`	Footer for a section or page
`<figure>`	Self-contained content (images, diagrams, etc.)
`<figcaption>`	Caption for `<figure>`
`<mark>`	Highlighted/marked text
`<time>`	Dates and times
`<address>`	Contact information

Example: Semantic Page Structure

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <title>Semantic HTML Example</title>
    </head>
    <body>
        <header>
```

```
            <h1>My Blog</h1>
            <nav>
                <ul>
                    <li><a href="#home">Home</a></li>
                    <li><a href="#about">About</a></li>
                </ul>
            </nav>
        </header>

        <main>
            <article>
                <header>
                    <h2>Semantic HTML Explained</h2>
                    <p><time datetime="2024-06-01">June 1, 2024</time></p>
                </header>
                <section>
                    <p>Semantic HTML makes your code more meaningful...</p>
                </section>
                <footer>
                    <p>Written by <address>Jane Doe</address></p>
                </footer>
            </article>
            <aside>
                <h3>Related Links</h3>
                <ul>
                    <li><a href="#">HTML5 Guide</a></li>
                </ul>
            </aside>
        </main>

        <footer>
            <p>&copy; 2024 My Blog</p>
        </footer>
    </body>
</html>
```

When to Use Semantic Elements

- Use `<header>`, `<footer>`, `<nav>`, and `<main>` for page layout.
- Use `<section>` for grouping related content.
- Use `<article>` for independent, reusable content (blog posts, news, etc.).
- Use `<aside>` for sidebars or extra info.
- Use `<figure>` and `<figcaption>` for images with captions.

Tip: Avoid using non-semantic tags like `<div>` and `<span>` for structure when a semantic element is available.

14. Multimedia

HTML supports embedding multimedia content such as audio and video directly into web pages using the `<audio>` and `<video>` elements.

Audio

The `<audio>` tag is used to embed sound content.

Basic Syntax:

```
<audio src="audio-file.mp3" controls>
    Your browser does not support the audio element.
</audio>
```

- `src` : Path to the audio file.
- `controls` : Displays play, pause, and volume controls.

Example with Multiple Sources:

```
<audio controls>
    <source src="audio.mp3" type="audio/mpeg">
    <source src="audio.ogg" type="audio/ogg">
    Your browser does not support the audio element.
</audio>
```

Video

The `<video>` tag is used to embed video content.

Basic Syntax:

```
<video src="video-file.mp4" controls width="400">
    Your browser does not support the video tag.
</video>
```

- `src` : Path to the video file.
- `controls` : Shows video controls.

Example with Multiple Sources and Poster:

```
<video width="400" controls poster="thumbnail.jpg">
```

```
    <source src="movie.mp4" type="video/mp4">
    <source src="movie.webm" type="video/webm">
    Your browser does not support the video tag.
</video>
```

- `poster`: Image shown before the video plays.

Embedding YouTube or Other Videos

Use the `<iframe>` tag to embed videos from platforms like YouTube:

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/VIDEO_ID" frameborder="0" allowfullscreen></iframe>
```

Common Attributes for `<audio>` and `<video>`

Attribute	Description	Example Usage
`src`	Path or URL to the media file	`<audio src="song.mp3">`
`controls`	Displays playback controls (play, pause, etc.)	`<video controls>`
`autoplay`	Starts playing automatically when loaded	`<audio autoplay>`
`loop`	Repeats the media after it ends	`<video loop>`
`muted`	Mutes the audio by default	`<video muted>`
`preload`	Hints how much media to preload (`auto` , `metadata` , `none`)	`<audio preload="auto">`
`poster`	(Video only) Image shown before playback starts	`<video poster="thumb.jpg">`
`width`	(Video only) Sets the width of the video player	`<video width="400">`
`height`	(Video only) Sets the height of the video player	`<video height="300">`

Tip: Always provide fallback text for unsupported browsers and use multiple sources for better compatibility.

15. HTML Entities

HTML entities are special codes used to display reserved characters, symbols, or characters not available on the keyboard. They start with `&` and end with `;`.

Why Use HTML Entities?

- To display reserved characters (like `<`, `>`, `&`) that would otherwise be interpreted as HTML.
- To show special symbols (©, ®, ™, etc.) or accented characters (é, ü, ñ).
- To ensure proper rendering across browsers and platforms.

Basic Syntax

```
&entity_name;
&#entity_number;
```

- **Named entity:** Uses a descriptive name (e.g., `©` for ©).
- **Numeric entity:** Uses a Unicode or ASCII code (e.g., `©` for ©).

Common HTML Entities

Character	Named Entity	Numeric Entity	Description
<	`<`	`<`	Less-than
>	`>`	`>`	Greater-than
&	`&`	`&`	Ampersand
"	`"`	`"`	Double quote
'	`'`	`'`	Single quote
©	`©`	`©`	Copyright
®	`®`	`®`	Registered
™	`™`	`™`	Trademark
	` `	` `	Non-breaking space
£	`£`	`£`	Pound Sterling
€	`€`	`€`	Euro
¥	`¥`	`¥`	Yen
§	`§`	`§`	Section
¶	`¶`	`¶`	Paragraph

Example Usage

```
<p>5 &lt; 10 &amp;amp;&amp;amp; 10 &gt; 5</p>
<p>&copy; 2024 My Website &reg;</p>
<p>Use &quot;quotes&quot; and &apos;apostrophes&apos; in text.</p>
<p>Non-breaking space prevents line breaks.</p>
```

Special Characters and Symbols

HTML entities can be used for mathematical symbols, arrows, currency, Greek letters, and more. For a full list, see W3C HTML Entities Reference.

Tip: Use entities for reserved characters, invisible spaces, and special symbols to ensure correct display and avoid HTML parsing errors.

16. Iframes

Iframes (inline frames) allow you to embed another HTML document within the current document. They are commonly used for embedding videos, maps, and other web content.

Basic Syntax

```
<iframe src="URL" width="300" height="200"></iframe>
```

- `src` : The URL of the page to embed.
- `width` and `height` : The dimensions of the iframe.

Example Usage

```
<iframe src="https://www.example.com" width="600" height="400"></iframe>
```

Attributes

Attribute	Description
`src`	The URL of the page to embed.
`width`	The width of the iframe (in pixels or %).
`height`	The height of the iframe (in pixels or %).
`frameborder`	Specifies whether to display a border around the iframe (deprecated in HTML5).
`allowfullscreen`	Allows the iframe to be displayed in full-screen mode.

Security Considerations

- Be cautious when embedding content from untrusted sources, as iframes can be used for phishing attacks.
- Use the `sandbox` attribute to impose restrictions on the iframe content.

```
<iframe src="https://www.example.com" sandbox></iframe>
```

17. Comments and Favicon

HTML comments are used to add notes, explanations, or reminders within your code that are ignored by the browser and not displayed on the web page.

Comments help make your code more readable and maintainable, especially in large projects or when collaborating with others.

Basic Syntax

```
<!-- This is a comment -->
```

- Comments start with `<!--` and end with `-->` .
- Anything between these markers is treated as a comment.

Example Usage

```
<!-- Main navigation starts here -->
<nav>
    <!-- Navigation links -->
    <a href="#home">Home</a>
    <a href="#about">About</a>
</nav>

<!-- TODO: Add more sections below -->
```

Key Points

- Comments can be placed anywhere in the HTML document.
- They do not appear in the rendered web page but can be seen in the page source.
- Useful for temporarily disabling code during development:

```
<!-- <div class="ad-banner">Ad goes here</div> -->
```

- Comments cannot be nested (placing a comment inside another comment will break the comment).

Best Practices

- Use comments to explain complex code, mark sections, or leave notes for yourself or other developers.
- Avoid overusing comments for obvious code; keep them concise and relevant.
- Do not include sensitive information in comments, as they are visible in the page source.

Tip: Use comments to organize your HTML structure, such as marking the start and end of major sections.

Favicon in HTML

A **favicon** (short for "favorite icon") is a small icon associated with a website, typically displayed in the browser tab, bookmarks, and shortcuts. Adding a favicon improves your site's branding and user experience.

How to Add a Favicon

1. **Create a Favicon Image**
   - Common sizes: 16x16, 32x32, or 48x48 pixels.
   - Formats: `.ico` (most compatible), `.png`, `.svg`, `.gif`.
   - Name it `favicon.ico` for maximum compatibility.
2. **Place the Favicon in Your Project**
   - Put `favicon.ico` in your website's root directory (recommended).
   - For other formats or locations, specify the path in your HTML.
3. **Link the Favicon in HTML**

Add the following `<link>` tag inside the `<head>` section:

```
<!-- Standard favicon -->
<link rel="icon" type="image/x-icon" href="/favicon.ico">

<!-- PNG format -->
<link rel="icon" type="image/png" href="/favicon.png">

<!-- SVG format -->
<link rel="icon" type="image/svg+xml" href="/favicon.svg">
```

You can include multiple formats for broader browser support.

Example

```
<head>
    <title>My Website</title>
    <link rel="icon" type="image/x-icon" href="/favicon.ico">
    <link rel="icon" type="image/png" href="/favicon.png">
</head>
```

Advanced: Apple Touch Icon and More

For better support on mobile devices and platforms, add these tags:

```
<!-- Apple Touch Icon (iOS Home Screen) -->
<link rel="apple-touch-icon" sizes="180x180" href="/apple-touch-icon.png">

<!-- Android/Chrome -->
<link rel="icon" sizes="192x192" href="/android-chrome-192x192.png">
```

Tips

- Use a square image for best results.
- Clear your browser cache to see favicon changes.
- Use online tools like realfavicongenerator.net to generate all favicon sizes and HTML code.

18. HTML file path

File Paths in HTML

A **file path** specifies the location of a file (such as an image, CSS, or JavaScript file) that you want to include in your HTML document. Correctly specifying file paths is essential for linking resources.

Types of File Paths

1. Absolute Path

- Points to a file by its full URL or from the root directory.
- Starts with `/` (root-relative) or with a protocol (`http://`, `https://`).
- Example (root-relative):

```
<img src="/images/logo.png" alt="Logo">
```

- Example (full URL):

```
<a href="https://www.example.com/about.html">About</a>
```

2. Relative Path

- Points to a file relative to the current document's location.
- Does **not** start with `/` or a protocol.
- Example (same folder):

```
<img src="photo.jpg" alt="Photo">
```

- Example (subfolder):

```
<img src="images/photo.jpg" alt="Photo">
```

- Example (parent folder):

```
<img src="../logo.png" alt="Logo">
```

3. Current and Parent Directory

- `./` refers to the current directory (often omitted).
- `../` refers to the parent directory.

File Path Examples

Path Type	Example Path	Description
Absolute URL	`https://site.com/img.png`	Loads from a full web address
Root-relative	`/assets/img.png`	Loads from root of the website
Relative	`img.png`	Loads from same folder as HTML file
Relative (sub)	`images/img.png`	Loads from a subfolder
Relative (parent)	`../img.png`	Loads from parent folder

Best Practices

- Use **relative paths** for local resources to keep your site portable.
- Use **absolute URLs** for external resources (CDNs, other websites).
- Check your folder structure to avoid broken links.
- Use lowercase and avoid spaces in file/folder names for compatibility.

Tip: If your images or files are not showing, double-check the file path and spelling.

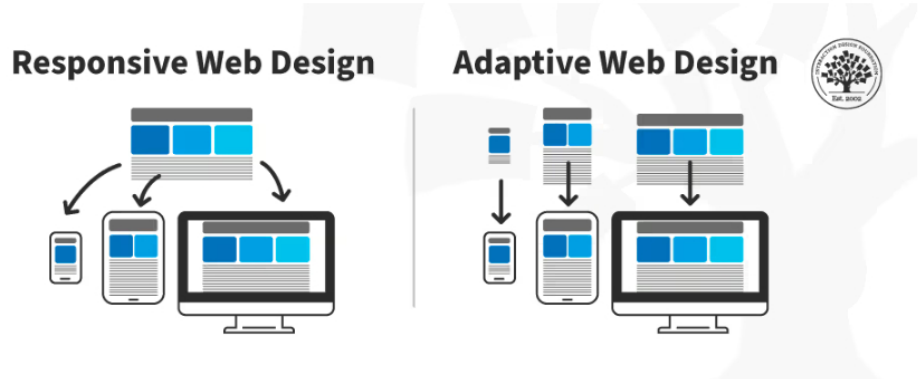
---

19. Responsive Design with HTML

Responsive design ensures your site works on all devices. Key HTML features:

- **Viewport meta tag:**

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```



- **Picture element for responsive images:**

```html
<picture>
    <source srcset="image-large.jpg" media="(min-width: 800px)">
    <img src="image-small.jpg" alt="Responsive Image">
</picture>
```

20. HTML Quotation and Citation Elements

HTML provides several elements for quotations and citations, helping to semantically mark up quoted or referenced content.

1. Inline Quotation: `<q>`

- Used for short, inline quotations.
- Browsers typically render with quotation marks.

Example:

```html
<p>He said, <q>HTML is easy to learn.</q></p>
```

2. Block Quotation: `<blockquote>`

- Used for longer quotations that are set apart from the main text.
- Usually indented by browsers.

Example:

```html
<blockquote cite="https://developer.mozilla.org/">
    HTML stands for HyperText Markup Language.
</blockquote>
```

- The optional `cite` attribute specifies the source URL.

3. Citation: `<cite>`

- Used to reference the title of a creative work (book, article, website, etc.).
- Typically rendered in italics.

Example:

```html
<p>Read more in <cite>HTML: The Definitive Guide</cite>.</p>
```

4. Abbreviation: `<abbr>`

- Marks up abbreviations or acronyms.
- The `title` attribute provides the full term on hover.

Example:

```html
<p><abbr title="Cascading Style Sheets">CSS</abbr> is used for styling.</p>
```

5. Definition: `<dfn>`

- Indicates the defining instance of a term.

Example:

```html
<p><dfn>HTML</dfn> is the standard markup language for web pages.</p>
```

6. Address: `<address>`

- Used for contact information, often for the author or owner of a document.

Example:

```html
<address>
    Contact: <a href="mailto:webmaster@example.com">webmaster@example.com</a>
</address>
```

Tip: Use these elements to improve document structure, accessibility, and search engine optimization.